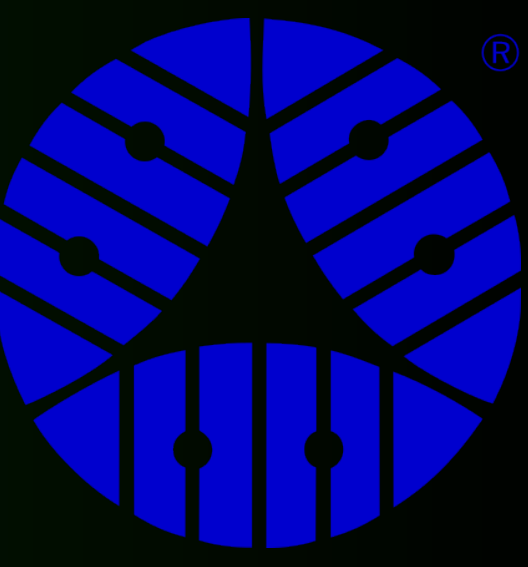




Wrocław
University
of Science
and Technology

Hybrid Neighborhood Evaluation in Tabu Search and Simulated Annealing for the Permutation Flow Shop Problem



Remigiusz Wojewódzki¹ Wojciech Bożejko¹

¹ Faculty of Information and Communication Technology, Wrocław University of Science and Technology, Poland

remigiusz.wojewodzki@gmail.com wojciech.bozejko@pwr.edu.pl

Introduction

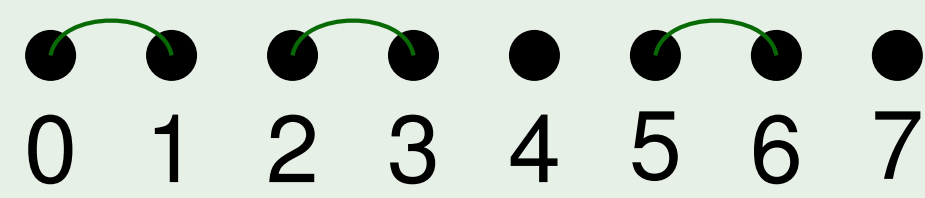
- The **Permutation Flow Shop Problem (PFSP)** schedules n jobs on m machines in the same order; the goal is to minimise the makespan $C_{\max}$. It is NP-hard for $m \geq 3$.
- Performance of metaheuristics for the PFSP depends not only on the framework but, equally importantly, on the choice of the **neighborhood structure** used at each iteration.
- We compare three classical neighborhoods of different granularity — Adjacent swap, the composite disjoint-adjacent neighborhood we call *Fibonacci*, and the recursive Dynasearch — inside Tabu Search (TS) and Simulated Annealing (SA).
- Experiments are conducted on the full Taillard benchmark with six time budgets between 100 ms and 10 s and the relative percentage deviation (PRD) of the obtained $C_{\max}$ to the best-known one as the performance metric.

The Fibonacci neighborhood

A move is a subset $S \subseteq \{1, \dots, n-1\}$ of adjacent-swap positions satisfying the **non-overlap** constraint

$$i \in S \Rightarrow i+1 \notin S,$$

and *all* selected swaps $\pi_i \leftrightarrow \pi_{i+1}$ are applied **simultaneously**.



The number of valid subsets in a permutation of length n equals the Fibonacci number F_{n+1} — hence the nickname. The **optimal** subset (minimal $\sum_{i \in S} \Delta_i$) is found by a single linear-time dynamic-programming pass given the precomputed $n-1$ adjacent-swap deltas.

Per-iteration cost

Neighborhood	Candidates	Total per iter.
Adjacent	$n - 1$ swaps	$O(mn)$
Fibonacci	$n - 1$ swaps + DP	$O(mn^2)$
Dynasearch (rec.)	$O(n^2)$ segments	$O(mn^3)$

The *Fibonacci* neighborhood explores an **exponentially-sized** set of moves (F_{n+1} valid subsets) at **quadratic** per-iteration cost — the sweet spot between cheap Adjacent and expensive Dynasearch.

Metaheuristic frameworks

Tabu Search (TS). Tabu list keyed by primitive moves or composite signatures; tenure $\tau_{\text{tabu}} = 10$; aspiration on global-best improvement.

Simulated Annealing (SA). Multiplicative cooling $T \leftarrow \alpha T$ with floor; **time-based reheating** kicks in after $\tau_{\text{stagn}} = 500$ ms of stagnation: $T \leftarrow \min(T_0, r \cdot T)$ with $r = 1.5$.

Both frameworks plug in the same set of neighborhoods, so any performance gap is attributable to the neighborhood, not the metaheuristic.

Key result — $n = 200, m = 20, tl = 5$ s, Tabu Search

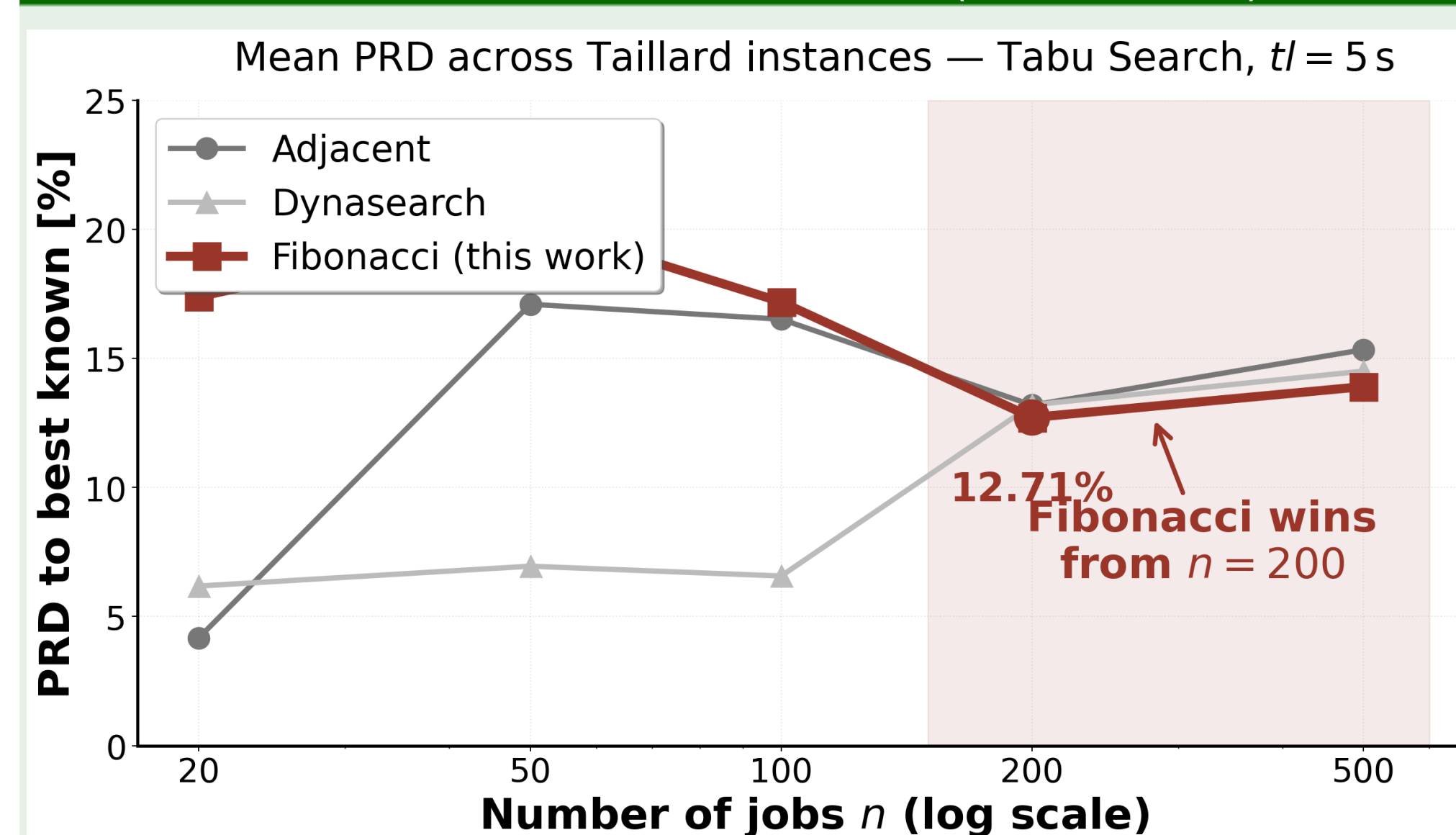
12.71 %
Fibonacci

13.20 %
Dynasearch

Adjacent = 13.19 %
Fibonacci = **12.71 %** (−0.48 pp vs. Adj, −0.49 pp vs. Dyn)
Dynasearch = 13.20 %

Fibonacci's compound move pays for itself once Dynasearch's $O(mn^3)$ cost no longer fits the time budget — the crossover happens at $n = 200$.

Where Fibonacci wins — PRD vs. n (TS, $tl = 5$ s)

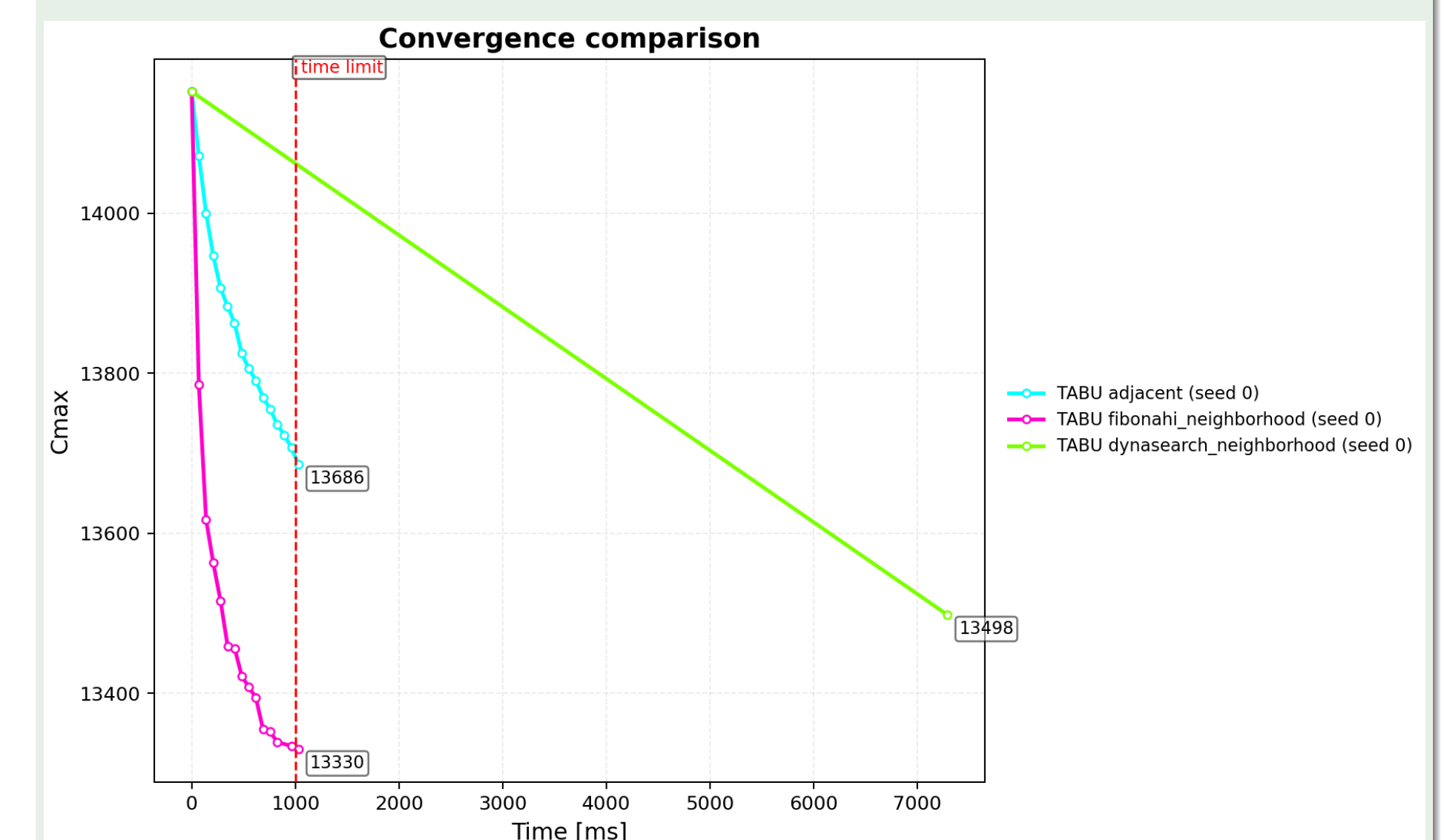


Below $n = 100$: Dynasearch's deep moves dominate.
From $n = 200$: Dynasearch cannot finish a single iteration within the budget, and Fibonacci's cheaper compound move pulls ahead and stays competitive at $n = 500$.

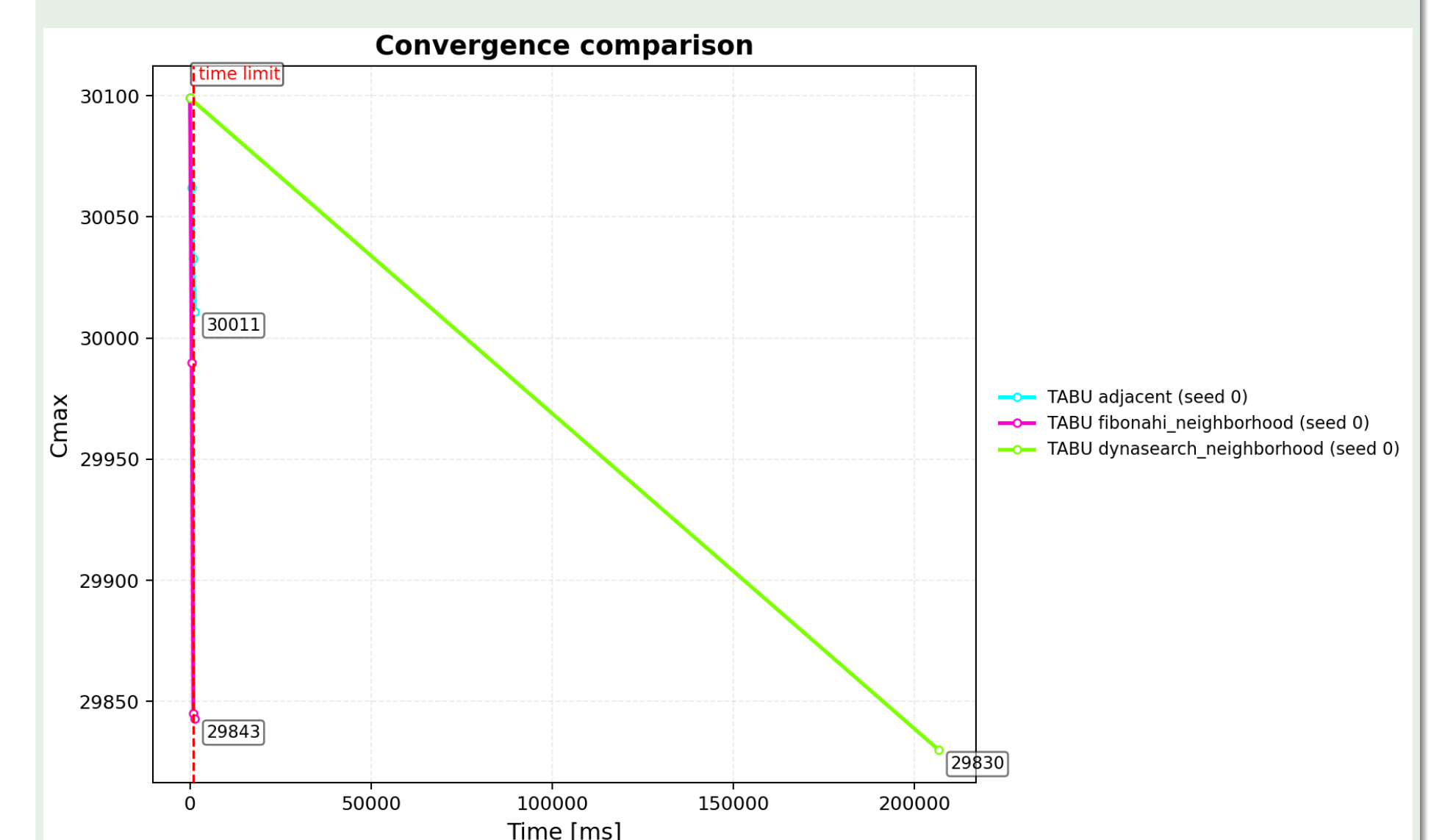
Per-instance numbers ($tl = 5$ s)

instance	n	m	fib _{TS}	fib _{SA}	dyn _{TS}	dyn _{SA}	adj _{TS}	adj _{SA}
tai20_5	20	5	17.39	17.39	6.18	6.18	4.17	6.24
tai50_10	50	10	20.23	20.23	6.95	6.95	17.10	9.72
tai100_10	100	10	17.19	17.19	6.57	6.48	16.52	8.27
tai200_10	200	10	12.71	12.71	13.20	13.20	13.19	8.53
tai200_20	200	20	18.60	18.60	20.71	20.71	19.83	20.20
tai500_20	500	20	13.91	13.90	14.51	14.51	15.34	15.82

Convergence example — $tl = 1$ s



tai200_20, $n = 200, m = 20$



tai500_20, $n = 500, m = 20$

Fibonacci (**magenta**) drops $C_{\max}$ in the first ~ 100 ms; Dynasearch (orange) barely finishes one iteration within the time budget.

Experimental setup

- Taillard instances, $n \in \{20, 50, 100, 200, 500\}$, $m \in \{5, 10, 20\}$ — 12 instance classes.
- Time budgets: $tl \in \{100, 500, 1000, 2000, 5000, 10000\}$ ms.
- SA: $T_0 = 1000$, $T_{\text{final}} = 1$, $\alpha = 0.95$, reheat $r = 1.5$, stagnation 500 ms.
- TS: tabu tenure $\tau_{\text{tabu}} = 10$, aspiration on global-best improvement.
- Platform: MacBook Pro M4 Pro, 24 GB RAM, Python 3.11.
- Metric: PRD to best-known $C_{\max}^*$, $\text{PRD} = \frac{C_{\max} - C_{\max}^*}{C_{\max}^*} \cdot 100\%$.

Conclusions

- Three neighborhood structures (Adjacent, *Fibonacci*, Dynasearch) compared inside TS and SA on the full Taillard benchmark.
- The composite *Fibonacci* neighborhood **dominates from $n = 200$ onward**: at $n = 200$ it improves over Adjacent by 0.48 pp and over Dynasearch by 0.49 pp at $tl = 5$ s; at $n = 500$ it remains competitive while Dynasearch cannot complete a single iteration.
- Dynasearch reaches deeper optima at small n but is impractical for $n \geq 200$ under short time budgets.
- SA's **time-based reheating** reliably escapes the plateau that the composite move induces once all $\Delta_i \geq 0$.

Future work

- Incremental delta evaluation for the composite move ($O(mn^2) \rightarrow O(mn)$).
- Adaptive switching between neighborhoods at runtime.
- Comparison against state-of-the-art FSP solvers (iterated local search, variable neighborhood descent).
- Quantum-assisted Fibonacci selection on D-Wave Advantage (windowed QUBO, n up to 500) — companion paper at SOCO 2026